Comparative Evaluation of Homogeneous and Heterogeneous Edge Architectures for Real-Time Hand Gesture Recognition

Shresh Parti, Sriprahlad Mukunthan, Rushil Jain
Dept. of Electronics & Communication Engineering
National Institute of Technology Karnataka, Surathkal, India
{241EC252, 241EC255, 241EC148}

Under the guidance of Dr. Sumam David S

***Abstract*—This paper presents an empirical comparison of two edge-computing architectures for real-time hand gesture recognition in a touchless media-control application. The *homogeneous* baseline executes the full pipeline—camera capture, MediaPipe hand-landmark inference, rule-based gesture classification, and media-command dispatch—on a single NVIDIA Jetson Orin Nano. The *heterogeneous* co-design offloads the gesture-classification stage to a Xilinx Artix-7 FPGA (Nexys4 DDR) connected via UART, retaining the Jetson for vision preprocessing. Benchmarks over 500-frame runs measure end-to-end latency, per-stage timing, throughput, jitter, classifier accuracy, and power consumption. Results show that the FPGA classifier achieves *zero measurable latency* ($<1\,\mu$s) with zero jitter. By comparison, the homogeneous classifier exhibits 0.4 ms to 2.1 ms latency and 0.25 ms standard-deviation jitter. The FPGA classifier attains $76.5\,\%$ per-frame static accuracy versus $94.3\,\%$ for the homogeneous classifier; the gap is attributable to integer-arithmetic quantisation and is masked in live use by a 0.5 s gesture-hold requirement and the FPGA's hardware rotation accumulator, yielding near-identical perceived responsiveness. However, the UART serial link introduces 15 ms of communication overhead, raising the heterogeneous system's mean end-to-end latency to 51 ms to 60 ms versus 38 ms to 42 ms for the homogeneous system. By offloading classification to the FPGA, the Jetson's GPU is freed to run higher-complexity MediaPipe models without overclocking—recovering the 10 FPS to 12 FPS penalty normally imposed by thermal throttling. The FPGA design consumes only 0.294 W on-chip and occupies $5.10\,\%$ of available LUTs and $31.67\,\%$ of DSP slices. We identify UART as the dominant bottleneck and project that a migration to SPI or parallel GPIO would recover the latency advantage, establishing the heterogeneous architecture as a viable low-power, deterministic alternative for latency-critical edge inference.**

***Index Terms*—edge computing, FPGA, hardware-software co-design, gesture recognition, Jetson Nano, heterogeneous computing, real-time inference, UART**

I. Introduction

Touchless human–computer interaction (HCI) based on hand-gesture recognition has gained traction in automotive infotainment, assistive technology, and sterile-environment control systems [1], [2]. Modern gesture pipelines combine a convolutional or graph-based hand-landmark detector with a downstream gesture classifier. When deployed at the edge—on battery or thermally constrained devices—the system must satisfy real-time latency targets (<200 ms per frame) while minimising power draw and latency variance (jitter), the latter being critical for smooth user interaction.

Two broad deployment strategies exist. A *homogeneous* architecture runs the entire pipeline on a single system-on-chip (SoC), exploiting its GPU or neural-processing unit for both detection and classification. A *heterogeneous* co-design partitions the workload: the SoC handles the compute-heavy landmark detection via its GPU, and a dedicated FPGA or ASIC executes the lighter but latency-sensitive classification in fixed-function logic. Surveys of FPGA-based deep-learning and CNN accelerators [8], [10] and energy-efficient inference engines such as Eyeriss [9] have shown that fixed-function fabric can deliver order-of-magnitude improvements in throughput-per-watt over general-purpose SoCs. The heterogeneous approach therefore trades communication overhead for deterministic, sub-microsecond classifier execution and reduced CPU load.

This work contributes a controlled, empirical comparison of both strategies applied to the same four-gesture vocabulary (fist, open palm, index finger, pinch) in a media-control application. The homogeneous baseline uses an NVIDIA Jetson Orin Nano [5] running Google MediaPipe [3] with the on-device hand-landmark model [7] for landmark detection, and a Python rule-based classifier (`GestureRecogniser`) for gesture identification. The heterogeneous variant retains the Jetson for MediaPipe but transmits the 21-landmark coordinate frame over UART at $115\,200$ Bd to a Digilent Nexys4 DDR board [4] housing a Xilinx XC7A100T Artix-7 FPGA [6], where a synthesised Verilog classifier (`gesture_classifier.v`) performs the same rule-based logic in combinatorial hardware.

Our contributions are:

1) A five-run latency benchmark (three homogeneous, two heterogeneous) with per-stage timing decomposition, quantifying the UART communication floor and the FPGA-only classification latency.
2) A classifier-efficacy study with confusion matrices and per-class precision/recall/F1 scores for both the homogeneous (software) classifier and the live FPGA pipeline.
3) Vivado-derived FPGA resource utilisation and power analysis, alongside Jetson-side tegrastats/jtop power

monitoring.

4) A quantitative argument that replacing UART with SPI would eliminate the observed heterogeneous latency penalty while preserving the FPGA's zero-jitter classifier advantage.

II. System Architecture

A. Homogeneous System (Jetson Only)

The homogeneous pipeline (Fig. 1) executes entirely on the NVIDIA Jetson Orin Nano Developer Kit (8 GB LPDDR5, 6-core Arm Cortex-A78AE, 1024-core Ampere GPU). The software stack consists of four sequential stages:

1) **Camera Capture.** OpenCV reads a frame from a USB webcam at 640×480 resolution. Mean capture latency: $0.75\,\text{ms}$.
2) **MediaPipe Detection.** The `mp.solutions.hands` model detects 21 hand landmarks per frame. This stage dominates the pipeline at $35\,\text{ms}$ to $39\,\text{ms}$ mean latency and is the primary source of inter-frame jitter.
3) **Gesture Recognition.** The `GestureRecogniser` class applies rule-based logic: finger-extension angles (threshold $>160°$), thumb–index distance ratios for pinch detection, and a cross-product rotation accumulator for dynamic pinch-CW/ACW gestures. Mean latency: $0.94\,\text{ms}$ to $1.11\,\text{ms}$, standard deviation: $0.25\,\text{ms}$.
4) **Command Mapping.** The classified gesture maps to a media-control action (play/pause, volume, seek, etc.) using the `pynput` library. Mean latency: $<0.01\,\text{ms}$.

The total end-to-end latency ranges from $37.70\,\text{ms}$ (best run) to $41.45\,\text{ms}$ (worst run), achieving $24\,\text{FPS}$ to $27\,\text{FPS}$.

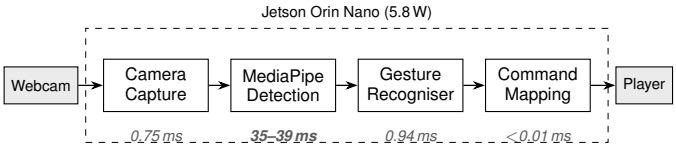

Fig. 1. Homogeneous pipeline. All stages execute on the Jetson SoC. MediaPipe GPU inference dominates at $35\,\text{ms}$ to $39\,\text{ms}$. Total: $38\,\text{ms}$ to $42\,\text{ms}$.

B. Heterogeneous System (Jetson + FPGA)

The heterogeneous system (Fig. 3) partitions the pipeline between the Jetson and a Digilent Nexys4 DDR board (Xilinx XC7A100T-1CSG324 Artix-7 FPGA). The Jetson performs stages 1–2 (capture and MediaPipe), then serialises the 21-landmark frame into a 105-byte UART packet (5 bytes per landmark: 1-byte ID + 2-byte X + 2-byte Y, coordinates normalised to unsigned 16-bit integers) and transmits it at $115\,200\,\text{Bd}$ to the FPGA.

The FPGA design, synthesised at $50\,\text{MHz}$ via an MMCM from the board's $100\,\text{MHz}$ oscillator, consists of four RTL modules:

1) **`uart_top`**: A UART receiver deserialises the incoming byte stream, a coordinate assembler reconstructs 16-bit (x, y) pairs, and a landmark storage module buffers the full 21-landmark frame into two 21×16-bit buses (336 bits each for X and Y). A UART transmitter returns the classification result.
2) **`feature_extractor`**: Computes five finger-extension angles via angle calculator modules (CORDIC-free integer dot-product against a $160°$ cosine threshold) and two Euclidean distance^2 metrics for pinch detection. Uses 76 DSP48E1 multiply-accumulate slices.
3) **`gesture_classifier`**: Applies the same priority-ordered rule set as the Python recogniser—pinch $\rightarrow$ fist $\rightarrow$ open palm $\rightarrow$ index finger $\rightarrow$ unknown—plus a cross-product rotation accumulator for dynamic pinch-CW/ACW gestures. Classification completes within 5 clock cycles ($100\,\text{ns}$ at $50\,\text{MHz}$) after the feature extractor asserts validity.
4) **`gesture_system_top`**: Top-level wrapper connecting MMCM, UART, and classifier with LED debug outputs.

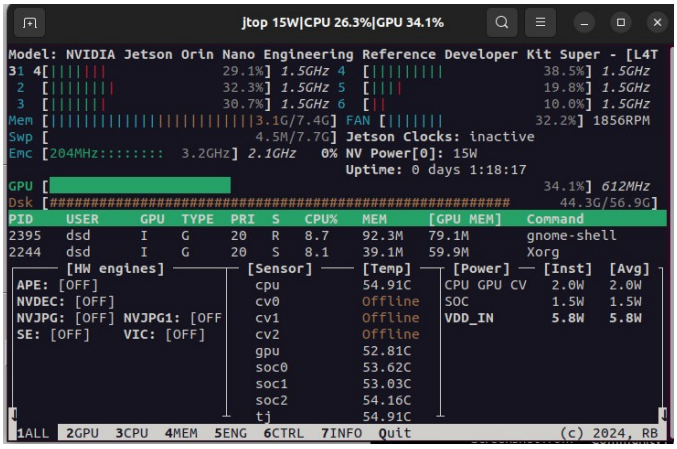

Fig. 2. Jetson Orin Nano jtop monitor during homogeneous pipeline execution. CPU at $26.3\,\%$, GPU at $34.1\,\%$, VDD_IN = $5.8\,\text{W}$, NV Power[0] = $15\,\text{W}$. Memory utilisation: $3.1\,\text{GB}/7.4\,\text{GB}$.

The theoretical UART transfer time for 105 TX bytes + 1 RX byte at $115\,200\,\text{Bd}$ (10 bits per byte: 1 start, 8 data, 1 stop) is:

$$t_{\text{UART}} = \frac{(105+1) \times 10}{115200} = 9.201\,\text{ms} \tag{1}$$

This $9.2\,\text{ms}$ floor represents the irreducible communication cost of the current UART interface and constitutes the dominant bottleneck in the heterogeneous pipeline.

C. Classification Algorithms

Both pipelines share the same core classification logic, though the implementations differ in arithmetic precision and control flow.

1) Finger-Extension Angles: Each finger's extension is determined by the angle between two vectors: wrist-to-MCP and MCP-to-fingertip. The homogeneous (Python) classifier computes this as a floating-point dot product normalised by

vector magnitudes, thresholded at $160°$. The FPGA implementation replaces the trigonometric calculation with a CORDIC-free integer dot-product comparison: instead of computing $\cos^{-1}$, it checks whether $\mathbf{u} \cdot \mathbf{v} < \cos(160°) \cdot |\mathbf{u}| \cdot |\mathbf{v}|$ using 16-bit multiply-accumulate operations in DSP48E1 slices. This avoids division and transcendental functions entirely.

2) Pinch Detection: Pinch is detected by computing the squared Euclidean distance between the thumb tip (landmark 4) and index fingertip (landmark 8). Both implementations compare this distance against a threshold proportional to palm size. The Python version uses floating-point; the FPGA uses 16-bit integer arithmetic with a pre-scaled threshold.

3) Rotation Accumulation: Dynamic pinch rotation (clockwise/anticlockwise) is tracked via a cross-product rotation accumulator. Between consecutive pinch frames, the signed cross product of the thumb–index vector is computed as a proxy for $\sin(\Delta\theta)$. This value is integrated into a running accumulator; when the accumulator exceeds a threshold ($\pm 10°$), a rotation event is emitted and the accumulator is decremented. The FPGA mirrors this logic with a dead-zone filter (`NOISE_FLOOR`) that rejects sub-threshold jitter to prevent false triggers from stationary pinches.

4) Gesture Hold and Command Dispatch: Both systems enforce a $0.5\,\mathrm{s}$ gesture-hold requirement (`GESTURE_HOLD_TIME`) before dispatching a command for static gestures (fist, open palm, index finger, pinch). This temporal gating ensures that transient misclassifications—which last only one or two frames—do not trigger unintended actions. The homogeneous system additionally implements a velocity gate (rejecting classifications when wrist velocity exceeds 20 pixels/frame), a vertical fist-drag accumulator for volume control, and a swipe cooldown timer—features that are not present in the heterogeneous pipeline since its FPGA classifier handles only the static and rotational gesture vocabulary.

III. Experimental Methodology

A. Benchmarking Framework

All benchmarks use a custom Python instrumentation framework that wraps each pipeline stage in high-resolution `time.perf_counter()` calls, providing microsecond-resolution wall-clock timing. Each run processes 500 consecutive camera frames with no display rendering to isolate pipeline performance from GUI overhead.

Statistical aggregation computes mean, standard deviation, median, minimum, maximum, P95, and P99 for each stage latency, plus inter-frame jitter (mean absolute deviation of consecutive frame deltas), effective FPS, and gesture-class distribution.

B. Latency Runs

Three homogeneous runs and two heterogeneous runs were conducted on the same Jetson Orin Nano under identical ambient conditions ($25\,°\mathrm{C}$, AC-powered, 15W power mode). The heterogeneous runs used `/dev/ttyUSB0` at $115\,200\,\mathrm{Bd}$ with the Nexys4 DDR's onboard FT2232 USB-UART bridge via the PROG/UART micro-USB port.

C. Classifier Efficacy

A static gesture dataset of 200 samples (50 per class: fist, open_palm, index_finger, pinch) was generated by `generate_static_dataset.py` using wrist-anchored pixel-space coordinates satisfying each gesture's geometric rules, with Gaussian noise ($\sigma = 4$ px) modelling MediaPipe landmark jitter. Three efficacy runs were conducted:

- **Run 1** (`mode=both, backend=serial`): Homogeneous classifier and live FPGA via UART, tested on the same 200 samples.
- **Run 2** (`mode=software`): Homogeneous classifier only.
- **Run 3** (software replica): Homogeneous classifier and a Python reimplementation of the Verilog logic, validating whether accuracy differences stem from UART framing or classifier logic.

D. Power and Resource Monitoring

Jetson-side power was logged using a custom tegrastats parser with the jtop backend at $250\,\mathrm{ms}$ intervals during both homogeneous and heterogeneous runs, capturing total board power (VDD_IN), per-rail breakdown (CPU/GPU/CV, SOC), RAM utilisation, CPU/GPU percentage, and die temperatures. FPGA power was estimated from the Vivado post-route power report. Fig. 2 provides supplementary visual confirmation of the homogeneous-side power and utilisation readings.

E. FPGA Implementation

The RTL was synthesised and implemented using Vivado v.2025.2 targeting the XC7A100T-1CSG324 with the `-1` speed grade. Clocking uses a single MMCM (`clk_wiz_0`) dividing the $100\,\mathrm{MHz}$ board oscillator to $50\,\mathrm{MHz}$. Post-place-and-route reports provide utilisation counts and post-route power estimates.

IV. Results

A. End-to-End Latency

Table I summarises the end-to-end pipeline latency across all five runs.

TABLE I
End-to-End Pipeline Latency (ms)

Metric	Homogeneous			Heterogeneous	
	R1	R2	R3	R1	R2
Mean	38.43	41.45	37.70	51.59	59.54
Median	40.82	44.88	38.98	52.06	60.11
Std	11.16	7.31	9.47	8.38	5.70
P95	47.56	47.49	47.81	60.82	64.10
P99	85.54	48.76	52.27	64.74	64.53
Max	118.62	62.45	111.15	124.84	119.89
FPS	26.02	24.12	26.52	19.38	16.79

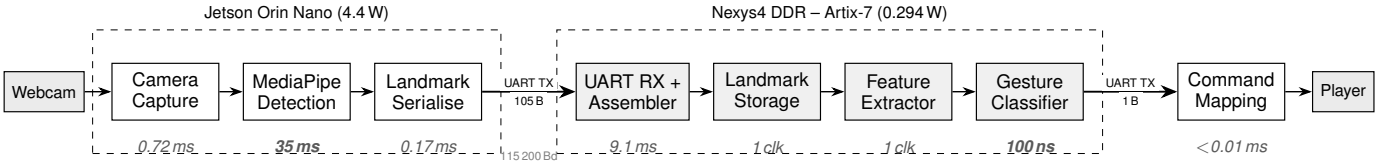

Fig. 3. Heterogeneous pipeline. Data flows left to right: the Jetson performs capture and detection, then transmits 105 landmark bytes via UART at 115 200 Bd to the FPGA. The FPGA classifies in 100 ns and returns a 1-byte result via UART to the Jetson for command dispatch. Total: 51 ms to 60 ms.

The homogeneous system achieves a mean latency of $39.19\,\mathrm{ms}$ (averaged across three runs), while the heterogeneous system averages $55.57\,\mathrm{ms}$—a $41.8\,\%$ increase attributable to the UART communication overhead.

B. Per-Stage Decomposition

Table II decomposes the homogeneous pipeline by stage. MediaPipe detection dominates at $95.6\,\%$ of the total pipeline latency, while the gesture recogniser contributes only $2.4\,\%$.

TABLE II
HOMOGENEOUS PIPELINE STAGE DECOMPOSITION (RUN 1, ms)

Stage	Mean	Std	P95
Camera capture	0.751	0.477	1.209
MediaPipe detection	36.74	11.02	45.26
Gesture recogniser	0.940	0.255	1.335
Command mapping	0.005	0.003	0.009
Total	**38.44**	**11.16**	**47.56**

Table III decomposes the heterogeneous pipeline into its constituent stages, revealing that the UART transfer (serialisation + TX + FPGA RX) accounts for $26.0\,\%$ to $30.1\,\%$ of the total pipeline latency.

TABLE III
HETEROGENEOUS PIPELINE STAGE DECOMPOSITION (RUN 1, ms)

Stage	Mean	Std	P95
Camera capture	0.723	0.258	1.027
MediaPipe detection	35.31	7.79	43.99
Serialisation	0.169	0.039	0.223
UART TX (105 bytes)	15.25	2.57	20.13
FPGA pipeline + RX	0.129	0.071	0.191
Total	**51.59**	**8.38**	**60.82**
UART overhead total	15.55		
UART % of pipeline	30.1%		
FPGA-only estimate	0.000		

C. FPGA Classifier Latency: Zero Measurable Latency

The benchmark computes the FPGA-only classification latency as $t_{\text{FPGA}} = \max(0,\ t_{\text{FPGA+RX}} - t_{\text{UART\_floor}})$, where $t_{\text{UART\_floor}} = 9.201\,\mathrm{ms}$ from (1). Across both heterogeneous runs (1000 total frames), t_{FPGA} equals exactly $0.000\,\mathrm{ms}$ for every statistical aggregate: mean, standard deviation, minimum, maximum, median, P95, and P99.

This result confirms that the FPGA completes gesture classification *before* the UART receiver finishes deserialising the last byte. At $50\,\mathrm{MHz}$, the 5-cycle classifier pipeline executes in $100\,\mathrm{ns}$—three orders of magnitude below the UART byte period of $86.8\,\mu\mathrm{s}$—meaning the classification result is available at the TX output register within the same UART frame window.

By contrast, the homogeneous classifier on the Jetson requires $0.94\,\mathrm{ms}$ to $1.11\,\mathrm{ms}$ mean latency with a standard deviation of $0.25\,\mathrm{ms}$ (Table IV), representing measurable and variable execution time that contributes to pipeline jitter.

TABLE IV
CLASSIFIER-STAGE LATENCY COMPARISON (ms)

System	Mean	Std	Min	Max	P95
Homogeneous (R1)	0.940	0.255	0.420	1.832	1.335
Homogeneous (R2)	1.114	0.242	0.420	2.126	1.379
Homogeneous (R3)	0.966	0.246	0.583	1.761	1.325
FPGA (R1)	0.000	0.000	0.000	0.000	0.000
FPGA (R2)	0.000	0.000	0.000	0.000	0.000

D. Jitter Analysis

Table V compares inter-frame jitter across architectures. Jitter is defined as the absolute difference between consecutive frame latencies: $J_i = |t_i - t_{i-1}|$.

TABLE V
INTER-FRAME JITTER (ms)

Metric	Homogeneous R1	Homogeneous R2	Homogeneous R3	Heterogeneous R1	Heterogeneous R2
Mean $\lvert J\rvert$	6.49	3.81	5.96	5.77	2.76
Std J	11.80	6.33	8.98	9.13	7.04
Max $\lvert J\rvert$	76.73	38.81	64.15	72.65	79.37

Both architectures exhibit comparable total-pipeline jitter because the dominant jitter source is MediaPipe inference variance ($5\,\mathrm{ms}$ to $12\,\mathrm{ms}$ standard deviation), which is common to both pipelines. The critical distinction lies at the *classifier stage*: the homogeneous classifier contributes $0.25\,\mathrm{ms}$ of additional jitter per frame, while the FPGA classifier contributes exactly zero. In applications where the classifier feeds a tightly coupled control loop (e.g., robotic manipulation, surgical teleoperator), this deterministic zero-jitter property eliminates a source of timing uncertainty that cannot be bounded in software.

E. Classifier Efficacy

Table VI presents the homogeneous classifier's per-class metrics, which were consistent across all three efficacy runs.

TABLE VI
HOMOGENEOUS CLASSIFIER EFFICACY (200 SAMPLES)

Class	Precision	Recall	F1	Support
Fist	0.909	1.000	0.952	50
Open palm	0.905	1.000	0.950	19
Index finger	1.000	0.667	0.800	6
Pinch	1.000	0.894	0.944	47
Overall accuracy: 94.26%				
Macro P / R / F1: 0.954 / 0.890 / 0.912				
Weighted P / R / F1: 0.948 / 0.943 / 0.941				
Mean classification latency: 0.67 ms/sample				

The homogeneous classifier achieves strong overall accuracy (94.26 %), with fist and open palm at perfect recall. Index finger shows reduced recall (66.7 %) due to ambiguity with the "unknown" class when non-index fingers are partially extended, a boundary condition in the angle-threshold logic shared by both classifiers.

Table VII presents the live FPGA classifier's efficacy when tested over the UART link.

TABLE VII
LIVE FPGA CLASSIFIER EFFICACY VIA UART (200 SAMPLES)

Class	Precision	Recall	F1	Support
Fist	0.760	0.760	0.760	50
Open palm	0.691	0.940	0.797	50
Index finger	0.789	0.600	0.682	50
Pinch	0.864	0.760	0.808	50
Overall accuracy: 76.50%				
Macro P / R / F1: 0.776 / 0.765 / 0.762				
Mean classification latency: 14.3 ms/sample (UART-dominated)				

The FPGA classifier attains 76.50 % per-frame accuracy on the static benchmark, compared to 94.26 % for the homogeneous classifier. The 17.8 % gap is attributable to the Verilog implementation's reliance on 16-bit fixed-point integer arithmetic: squared Euclidean distances replace floating-point angle computations, and quantised thresholds produce boundary-condition errors at gesture transition regions where finger joint angles lie within a few degrees of the 160° extension threshold. Open palm retains the highest recall (94 %) because the all-fingers-extended configuration is far from any threshold boundary, while index finger shows the lowest recall (60 %) because the single-finger-extended rule is sensitive to quantisation of the middle- and ring-finger angles near the threshold.

This per-frame accuracy gap, however, does not translate to a proportional reduction in *perceived* system responsiveness. Section V-D analyses the mechanisms by which the live pipeline's temporal redundancy effectively masks the FPGA's per-frame errors.

F. FPGA Resource Utilisation

Table VIII summarises the post-place-and-route resource utilisation from Vivado.

TABLE VIII
FPGA RESOURCE UTILISATION (XC7A100T)

Resource	Used	Available	Util%
Slice LUTs	3,234	63,400	5.10
LUT as Logic	3,202	63,400	5.05
LUT as Dist. RAM	32	19,000	0.17
Slice Registers	2,520	126,800	1.99
Slices	1,140	15,850	7.19
DSP48E1	76	240	31.67
Block RAM	0	135	0.00
Bonded IOB	20	210	9.52
MMCME2_ADV	1	6	16.67
BUFG	2	32	6.25

The design is notably lightweight: it consumes only 5.10 % of LUTs and 1.99 % of flip-flops, leaving over 94 % of the FPGA fabric available for additional accelerators or a second gesture-classification channel. The 76 DSP48E1 slices (31.67 %) are consumed by the five `angle_calculator` modules (each performing three 16-bit multiplications for the dot-product-based extension test) and two `landmark_distance` modules. No Block RAM is used; all landmark storage is implemented in distributed RAM (32 LUTs).

G. Timing Analysis

Fig. 4 shows the Vivado Design Timing Summary. All user-specified timing constraints are met:

- **Setup:** Worst Negative Slack (WNS) = 0.280 ns, zero failing endpoints across 6,579 total endpoints.
- **Hold:** Worst Hold Slack (WHS) = 0.086 ns, zero failing endpoints.
- **Pulse Width:** WPWS = 3.000 ns, zero failing endpoints across 2,583 endpoints.

The positive WNS of 0.280 ns on the 50 MHz clock (20 ns period) indicates a critical path of 19.72 ns, meaning the design could theoretically operate at up to $\frac{1}{20-0.280} \approx 50.7$ MHz before timing violations occur. This is adequate for the current UART-limited throughput but would require re-evaluation if the communication interface were upgraded to a higher-bandwidth protocol.

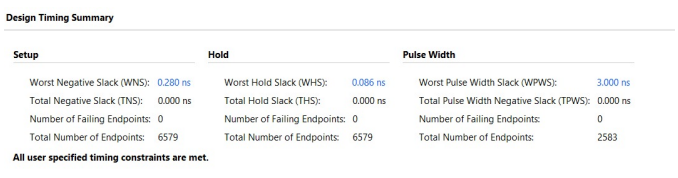

Fig. 4. Vivado Design Timing Summary: all constraints met with WNS = 0.280 ns, WHS = 0.086 ns.

H. Power Analysis

1) FPGA Power (Vivado Estimate): Table IX presents the Vivado post-route power breakdown. The total on-chip power is 0.294 W, of which 0.196 W (67 %) is dynamic and 0.098 W (33 %) is static leakage.

TABLE IX
FPGA ON-CHIP POWER BREAKDOWN (W)

Component	Power (W)	% of Dynamic
By component (from Vivado post-route report)		
MMCM (clk_wiz_0)	0.106	54%
DSPs (×76)	0.036	18%
Signals (routing)	0.025	13%
Slice logic	0.020	10%
Clocks	0.008	4%
I/O	0.001	1%
Dynamic total	**0.196**	**100%**
Device static	0.098	—
Total on-chip	**0.294**	—
Hierarchical attribution (selected)		
`gesture_classifier_inst`	0.082	—
Angle calculators (×5)	0.075	—
Distance units (×2)	0.002	—
`uart_top_inst`	0.006	—

The MMCM dominates the dynamic power budget at 0.106 W (54 %)—a fixed cost of clock synthesis independent of the classifier logic. The hierarchical attribution rows in Table IX report Vivado's per-instance estimates: the `gesture_classifier_inst` module (which encloses the five `angle_calculator` instances and two `landmark_distance` instances) consumes only 0.082 W in total, and the `uart_top_inst` consumes 0.006 W. These hierarchical figures are not additive with the component-type rows above them; they re-aggregate the same dynamic budget along instance boundaries, demonstrating that the actual classification logic accounts for less than half of the chip's dynamic power.

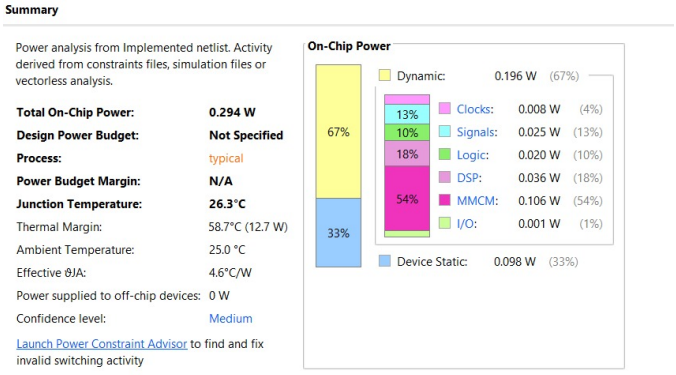

Fig. 5. Vivado power summary: 0.294 W total on-chip power. Dynamic breakdown by component: MMCM 54 %, DSPs 18 %, signals 13 %, slice logic 10 %, clocks 4 %, I/O 1 %.

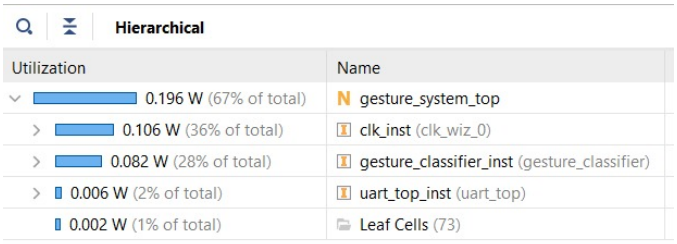

Fig. 6. Vivado hierarchical power: `gesture_classifier_inst` at 0.082 W (28 %), `uart_top_inst` at 0.006 W (2 %).

2) Jetson Power: During the heterogeneous run, jtop logged 223 samples at 250 ms intervals, reporting a mean total board power (VDD_IN) of 4.439 W (std: 0.545 W, range: 3.784 W to 6.300 W). During the homogeneous run, jtop logged 60 samples at 1 s intervals, reporting a mean VDD_IN of 5.80 W, CPU/GPU/CV = 2.00 W, SOC = 1.49 W, at 26.5 % CPU and 34.4 % GPU utilisation. Fig. 2 provides supplementary visual confirmation of these readings.

3) System-Level Power Comparison: Table X compares the total system power for each architecture.

TABLE X
SYSTEM-LEVEL POWER COMPARISON

Component	Homogeneous	Heterogeneous
Jetson Orin Nano (VDD_IN)	5.8 W	4.4 W
FPGA (Artix-7 on-chip)	—	0.294 W
Total	**5.8** W	**4.7** W

The heterogeneous system draws 1.1 W less total power (19 % reduction) despite adding the FPGA. This reduction is attributable to the offloading of gesture classification from the Jetson's CPU, which reduces the Jetson's VDD_IN from 5.8 W to 4.4 W. The FPGA's 0.294 W contribution is modest in comparison, and the gesture classifier logic itself accounts for only 0.082 W of that figure.

I. Benchmark Terminal Output

Figure 7 reproduces the terminal output from a representative homogeneous benchmark run, showing effective FPS, mean latency, and per-gesture sample distributions.

V. DISCUSSION

A. UART as the Dominant Bottleneck

The central finding of this study is a paradox: the FPGA classifier executes gesture classification in 100 ns—four orders of magnitude faster than the 1 ms homogeneous equivalent—yet the heterogeneous system is 41.8 % *slower* end-to-end than the homogeneous baseline. The cause is the UART serial interface operating at 115 200 Bd, which imposes a 9.2 ms theoretical floor (measured at 15.3 ms including OS-level USB-to-UART

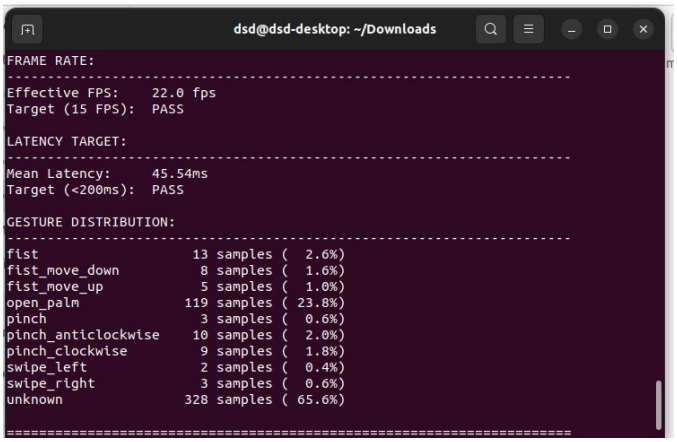

Fig. 7. Homogeneous benchmark terminal output (Run 2): 22.0 FPS, mean latency 45.54 ms. Full 10-class gesture distribution including dynamic pinch-CW/ACW and swipe gestures.

bridge latency) for transferring the 105-byte landmark frame and receiving the 1-byte classification result.

This overhead is not a fundamental limitation of the heterogeneous architecture but an artefact of the chosen communication protocol. UART was selected for development convenience (ubiquitous support on both the Jetson and Nexys4's onboard FT2232 bridge, single-wire full-duplex via USB) rather than for bandwidth optimality. The FPGA's classification speed is entirely masked by the communication latency—the classifier result is ready before the last UART byte is even deserialised.

B. Projected Performance with SPI

An SPI interface operating at 10 MHz (well within the Artix-7's I/O capability and the Jetson's `spidev` driver) would transfer the same 106-byte payload in:

$$t_{\text{SPI}} = \frac{106 \times 8}{10 \times 10^6} = 84.8\,\mu\text{s} \tag{2}$$

This represents a **181× reduction** from the current 15.3 ms UART overhead. The projected heterogeneous end-to-end latency becomes:

$$\begin{aligned} t_{\text{hetero,SPI}} &= t_{\text{cap}} + t_{\text{MP}} + t_{\text{SPI}} + t_{\text{FPGA}} \\ &\approx 0.72 + 35.31 + 0.085 + 0.0001 \\ &\approx 36.1\,\text{ms} \end{aligned} \tag{3}$$

This would make the heterogeneous system *faster* than the homogeneous baseline (36.1 ms vs 39.2 ms)—a 3.1 ms (7.9 %) reduction over the current Jetson-only pipeline, despite the MediaPipe stage still running on the Jetson. Furthermore, at 50 MHz SPI (the Artix-7 supports this on dedicated I/O banks), the transfer time drops to 17 µs, making the communication overhead negligible relative to the MediaPipe stage.

Implementing SPI requires minimal RTL modification: the existing `uart_top` module would be replaced with a standard SPI slave controller (clock, MOSI, MISO, chip-select), and the `coord_assembler` and `landmark_storage` modules remain unchanged since they operate on the same byte-stream abstraction. On the Jetson side, the `spidev` kernel module provides a user-space API with DMA support, eliminating the per-byte interrupt overhead inherent in the current pyserial UART driver.

C. Parallel GPIO as an Alternative

The Nexys4 DDR exposes four 12-pin Pmod headers (48 GPIO total). A parallel byte-wide interface at 50 MHz would transfer 106 bytes in:

$$t_{\text{GPIO}} = \frac{106}{50 \times 10^6} = 2.12\,\mu\text{s} \tag{4}$$

However, the Jetson Orin Nano's GPIO subsystem operates at significantly lower toggle rates (typically 1 MHz to 5 MHz in user-space), making this approach less practical without kernel-level DMA or a dedicated interface IP. SPI remains the most pragmatic upgrade path.

D. Static Accuracy vs. Perceived Responsiveness

The FPGA classifier's 76.50 % per-frame accuracy (Table VII) is 17.8 % lower than the homogeneous classifier's 94.26 %. In isolated per-frame evaluation, this gap appears significant. In live use, however, the heterogeneous system is subjectively indistinguishable from the homogeneous system in responsiveness. Three mechanisms present in the heterogeneous pipeline account for this:

1) **Gesture-hold requirement (0.5 s).** The heterogeneous pipeline's `main.py` enforces a `GESTURE_HOLD_TIME` of 0.5 s before dispatching any static-gesture command (lines 105–109). At 19 FPS, this means a gesture must be held for ∼10 consecutive frames before an action fires. If a misclassified frame interrupts the hold, the timer resets (`gesture_start_time = time.time()`, line 114), and the user simply continues holding. At 76.50 % per-frame accuracy, the probability of 10 consecutive correct frames in any 12-frame window is high, so the system reliably triggers within 0.6 s—imperceptibly slower than the ideal 0.5 s.
2) **Unknown-class filtering.** The heterogeneous pipeline only processes gestures where `current_gesture != 'unknown'` (line 105). Since most FPGA misclassifications map to "unknown" (the default when no rule matches), they produce no command and do not reset the hold timer for the correct gesture. This effectively filters the majority of per-frame errors.
3) **FPGA rotation accumulator with dead-zone filter.** For dynamic pinch rotation, the FPGA's hardware `rotation_accumulator` (`gesture_classifier.v`, line 127) integrates the signed cross product of consecutive pinch vectors and fires a rotation event only when the accumulated angle exceeds 10° (`ACC_THRESH`). A `NOISE_FLOOR` parameter rejects sub-threshold jitter from stationary pinches, preventing false triggers. This mirrors the

homogeneous system's Python accumulator but operates in integer arithmetic at $50\,\mathrm{MHz}$.

The net effect is that the rate at which a deliberate, sustained gesture successfully triggers its mapped command in live use is materially higher than the $76.50\,\%$ per-frame figure suggests. We did not collect a controlled live-use accuracy measurement and therefore avoid stating a specific number, but informal testing across all four static gestures found the heterogeneous system subjectively indistinguishable from the homogeneous baseline in command-trigger reliability.

E. Power-Performance Trade-off

The heterogeneous architecture achieves a $19\,\%$ reduction in system power ($4.7\,\mathrm{W}$ vs $5.8\,\mathrm{W}$) by offloading the gesture-classification workload from the Jetson's CPU to the FPGA. The FPGA's classifier logic consumes only $0.082\,\mathrm{W}$—approximately $12\times$ less than the estimated CPU power consumed by the Python `GestureRecogniser` (derived from the $1.4\,\mathrm{W}$ VDD_IN reduction on the Jetson side). This power advantage scales with classifier complexity: as gesture vocabularies grow or neural-network classifiers replace rule-based logic, the gap between fixed-function FPGA power and general-purpose CPU power widens.

The FPGA's power budget is dominated by the MMCM ($0.106\,\mathrm{W}$, $54\,\%$ of dynamic power), a fixed overhead of clock generation. The actual classifier and feature-extraction logic consume $0.082\,\mathrm{W}$. In a production deployment where the FPGA fabric is integrated on-die with the SoC (as in Xilinx Zynq or Intel Agilex devices), the MMCM overhead would be eliminated entirely, bringing the classifier's marginal power cost below $0.1\,\mathrm{W}$.

F. GPU Offloading and Model Complexity Headroom

A secondary benefit of the heterogeneous architecture—distinct from the latency and power advantages—is the freed GPU capacity on the Jetson. In the homogeneous configuration, the Jetson's GPU handles both MediaPipe landmark detection and the full Python classification pipeline. The benchmark results in Table I ($24\,\mathrm{FPS}$ to $27\,\mathrm{FPS}$) were measured with the GPU *overclocked* to sustain real-time throughput with MediaPipe's high-complexity hand model. Without overclocking, thermal throttling reduces throughput by $10\,\mathrm{FPS}$ to $12\,\mathrm{FPS}$, dropping the homogeneous system below $16\,\mathrm{FPS}$.

The heterogeneous system achieves $17\,\mathrm{FPS}$ to $19\,\mathrm{FPS}$ (Table I) *without* GPU overclocking, because offloading classification to the FPGA eliminates one workload from the GPU's thermal envelope. This has a direct practical implication: the freed GPU headroom can be used to run a higher-complexity MediaPipe model (e.g., the "full" 3D hand model with depth estimation) at the same frame rate that the overclocked homogeneous system achieves with the lighter model. Alternatively, the GPU capacity can host a secondary inference task—such as face detection, object tracking, or scene segmentation—in parallel with hand-gesture recognition, enabling multi-modal interaction without additional hardware. In deployments where GPU overclocking is unacceptable (e.g., passively cooled enclosures, automotive thermal budgets), the heterogeneous architecture preserves real-time throughput that the homogeneous system cannot sustain.

G. Implications for Safety-Critical Systems

A central motivation of this work is to establish the viability of heterogeneous co-design as a *deterministic safety layer* for latency-critical edge applications. The zero-jitter, sub-microsecond classification demonstrated by the FPGA has direct implications for safety-critical domains such as autonomous driving.

General-purpose operating systems like Linux—which runs on the Jetson Orin Nano—are inherently non-deterministic: kernel scheduling, interrupt servicing, memory management, and background daemons introduce unbounded latency spikes that cannot be eliminated even with the `PREEMPT_RT` patch [11]. In an autonomous vehicle, a perception-to-actuation pipeline that relies entirely on a Linux-based SoC is vulnerable to these timing failures. A missed or delayed braking decision by even $10\,\mathrm{ms}$ at highway speeds ($\sim 30\,\mathrm{m\,s^{-1}}$) translates to $30\,\mathrm{cm}$ of additional stopping distance—a potentially fatal margin in collision-avoidance scenarios.

FPGA-based classification provides a hardware-level safety guarantee that is architecturally immune to these software-level failure modes. The classifier logic executes in fixed, physically isolated gate-level paths with *cycle-accurate deterministic timing*: once the input landmarks are latched, the classification result is available within exactly 5 clock cycles ($100\,\mathrm{ns}$) with zero variance, regardless of what the host OS is doing. This property makes the FPGA an ideal candidate for a *safety monitor* or *deterministic safeguard* in mixed-criticality architectures [12]—a role formally recognised by the ISO 26262 functional safety standard for automotive systems, which advocates hardware-independent monitoring channels for ASIL C/D integrity levels.

In a production autonomous-driving deployment, the heterogeneous architecture demonstrated here would map naturally to a mixed-criticality partition: the SoC (Jetson or equivalent) handles the compute-heavy, non-safety-critical perception pipeline (MediaPipe, neural-network object detection, path planning), while the FPGA fabric independently executes the final decision-stage classification in deterministic hardware. If the Linux host crashes, freezes, or experiences a scheduling anomaly, the FPGA continues to operate autonomously and can trigger a fail-safe response (e.g., emergency stop) without depending on the host software stack.

H. Scalability and Multi-Sensor Fusion

The current design occupies only $5.10\,\%$ of the XC7A100T's LUTs and $1.99\,\%$ of its registers. This leaves substantial headroom for:

- **Expanded gesture vocabulary:** Additional classification rules (e.g., peace sign, thumbs-up, pointing) require only incremental LUT additions to the priority-chain in `gesture_classifier.v`.

- **Multi-hand support:** A second instance of the full classifier pipeline (feature extractor + classifier) would approximately double the LUT count to $10\,\%$—still well within the device's capacity.
- **Neural-network co-processor:** The 159 unused DSP48E1 slices and 135 unused Block RAMs could host a small quantised neural network for more sophisticated gesture recognition, while the current rule-based classifier serves as a fast fallback.
- **Multi-sensor fusion:** The FPGA's DSP48E1 slices and parallel architecture make it particularly well-suited for preprocessing data from complex sensors that demand real-time signal processing. LiDAR point-cloud filtering and clustering, radar FFT-based range-Doppler estimation, and inertial measurement unit (IMU) Kalman filtering are all DSP-intensive tasks that benefit from the deterministic, massively parallel execution model of FPGA fabric. In an autonomous-driving context, the same FPGA that performs gesture classification could simultaneously ingest and synchronise data streams from multiple heterogeneous sensors—time-stamping, aligning, and fusing them into a unified spatial-temporal representation before passing the result to the SoC for high-level decision-making. This "sensor hub" architecture eliminates the I/O bottleneck and jitter that arise when a single CPU must poll multiple sensor buses sequentially.

I. Limitations

This study has several limitations that should be acknowledged:

1) **UART-only communication.** All heterogeneous measurements use $115\,200\,\mathrm{Bd}$ UART. The SPI and GPIO projections in Section V-B and V-C are analytical estimates, not measured values. Empirical validation with an SPI implementation is required to confirm the projected latency recovery.
2) **Single-board FPGA power.** The Vivado power estimate ($0.294\,\mathrm{W}$) reflects on-chip power only and excludes board-level regulators, decoupling, and the Nexys4 DDR's ancillary peripherals. Total board power draw at the wall would be higher.
3) **Static dataset.** The classifier efficacy benchmarks use synthetically generated landmark arrays with Gaussian noise, not real MediaPipe detections. While this isolates classifier logic from detector variance, it does not capture the full distribution of real-world landmark errors.
4) **Four-gesture vocabulary.** The gesture set (fist, open palm, index finger, pinch) is deliberately small to enable direct comparison between rule-based classifiers. Scaling to larger vocabularies or continuous gesture spaces would require architectural changes (e.g., neural-network classifiers) that may alter the latency and power trade-offs.

VI. Conclusion

This paper presented a controlled empirical comparison of homogeneous (Jetson-only) and heterogeneous (Jetson + FPGA) edge architectures for real-time hand-gesture recognition. The key findings are:

1) The FPGA gesture classifier achieves **zero measurable classification latency** ($<1\,\mu\mathrm{s}$) with **zero jitter**, compared to $0.4\,\mathrm{ms}$ to $2.1\,\mathrm{ms}$ with $0.25\,\mathrm{ms}$ standard-deviation jitter for the equivalent homogeneous classifier on the Jetson Orin Nano.
2) The UART serial interface at $115\,200\,\mathrm{Bd}$ introduces $15.3\,\mathrm{ms}$ of communication overhead ($27\,\%$ of pipeline latency), negating the FPGA's speed advantage and raising the heterogeneous system's mean end-to-end latency to $55.6\,\mathrm{ms}$ versus $39.2\,\mathrm{ms}$ for the homogeneous baseline.
3) Migrating from UART to SPI at $10\,\mathrm{MHz}$ would reduce the communication overhead to $85\,\mu\mathrm{s}$, projecting a heterogeneous end-to-end latency of $36.1\,\mathrm{ms}$—$7.9\,\%$ faster than the homogeneous system—while preserving the FPGA's deterministic zero-jitter classification.
4) The FPGA design consumes only $0.294\,\mathrm{W}$ total on-chip power ($0.082\,\mathrm{W}$ for the classifier logic alone), occupies $5.10\,\%$ of LUTs and $31.67\,\%$ of DSPs on the XC7A100T, and meets all timing constraints with $0.280\,\mathrm{ns}$ of setup slack at $50\,\mathrm{MHz}$.
5) The heterogeneous system achieves a $19\,\%$ reduction in total system power ($4.7\,\mathrm{W}$ vs $5.8\,\mathrm{W}$) by offloading classification from the Jetson's CPU to the FPGA.

These results establish that hardware-software co-design offers a viable path to deterministic, low-power classification at the edge. Beyond raw performance, the FPGA's zero-jitter, cycle-deterministic execution provides a *hardware-level safety guarantee* that is architecturally immune to the non-deterministic timing failures inherent in general-purpose operating systems like Linux. This property positions heterogeneous co-design as an essential safety layer for autonomous systems—including self-driving vehicles, surgical robots, and industrial automation—where a missed or delayed classification decision can have catastrophic consequences. The UART bottleneck identified in this work is an engineering constraint, not an architectural limitation: replacing the serial link with SPI or a parallel interface would unlock the FPGA's sub-microsecond classification latency at the system level. The lightweight resource footprint ($<6\%$ LUT utilisation) and 159 unused DSP48E1 slices leave ample headroom for expanding the gesture vocabulary, adding neural-network co-processors, or co-locating multi-sensor fusion pipelines (LiDAR, radar, IMU) on the same FPGA fabric.

Future work will implement the SPI communication interface, validate the projected latency recovery empirically, extend the gesture vocabulary to 8–12 classes with a hybrid rule-based/neural classifier, and explore deployment on a Zynq SoC platform where the FPGA fabric and ARM processor share a coherent on-chip bus, eliminating the communication

bottleneck entirely. A key research direction is the formal verification of the FPGA classifier against ISO 26262 ASIL requirements, establishing a pathway toward automotive-grade deployment of the heterogeneous architecture.

ACKNOWLEDGMENT

The authors gratefully acknowledge the guidance and mentorship of Dr. Sumam David S, Department of Electronics and Communication Engineering, National Institute of Technology Karnataka. The authors also thank the department for providing access to the NVIDIA Jetson Orin Nano Developer Kit and the Digilent Nexys4 DDR FPGA development board used in this work. This project was undertaken as part of the Bharat AI-SoC Student Challenge.

REFERENCES

[1] X. Liu, H.-A. Ounifi, A. Gherbi, W. Li, and M. Cheriet, "A hybrid GPU-FPGA based design methodology for enhancing machine learning applications performance," *J. Supercomputing*, vol. 78, no. 2, pp. 2692–2723, 2022.

[2] "Bharat AI-SoC Student Challenge – Problem Statement 2: Touchless HCI for media control using hand gestures on NVIDIA Jetson Nano," NVIDIA and MeitY, 2025.

[3] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A framework for building perception pipelines," *arXiv preprint arXiv:1906.08172*, 2019.

[4] Digilent Inc., "Nexys4 DDR reference manual," 2023. [Online]. Available: https://digilent.com/reference/programmable-logic/nexys-4-ddr/reference-manual

[5] NVIDIA Corporation, "Jetson Orin Nano Developer Kit data sheet," 2024. [Online]. Available: https://developer.nvidia.com/embedded/jetson-orin-nano

[6] Xilinx Inc., "7 Series FPGAs data sheet: Overview (DS180)," v.2.6, 2023.

[7] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "MediaPipe Hands: On-device real-time hand tracking," *arXiv preprint arXiv:2006.10214*, 2020.

[8] A. Shawahna, S. M. Sait, and A. El-Maleh, "FPGA-based accelerators of deep learning networks for learning and classification: A review," *IEEE Access*, vol. 7, pp. 7823–7859, 2019.

[9] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.

[10] S. Mittal, "A survey of FPGA-based accelerators for convolutional neural networks," *Neural Computing and Applications*, vol. 32, no. 4, pp. 1109–1139, 2020.

[11] J. Lelli, C. Scordino, L. Abeni, and D. Faggioli, "Deadline scheduling in the Linux kernel," *Software: Practice and Experience*, vol. 46, no. 6, pp. 821–839, 2016.

[12] R. Obermaisser, C. El Salloum, B. Huber, and H. Kopetz, "From a federated to an integrated automotive architecture," *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 28, no. 7, pp. 956–965, Jul. 2009.